REVISION NOTES • TOPIC 7

Nullability and Null Handling

Simple explanations • examples • Fundraising Platform • diagrams • interview preparation

Topic Coverage

`null` is one of the most important concepts to understand in professional C# development.

A large percentage of runtime bugs in object-oriented applications come from code assuming an object exists when it actually does not.

Consider:

C#

```
Gift? gift = null;

Console.WriteLine(
    gift.TotalAmount);
```

If `gift` is actually `null`, there is no `Gift` object from which C# can read:

EXAMPLE / FLOW

```
TotalAmount
```

and a runtime failure can occur.

Modern C# provides several features to make these problems easier to prevent:

EXAMPLE / FLOW

```
Nullable Value Types

Nullable Reference Types

Compiler Null-State Analysis

Null Checks

Null-Conditional Operators

Null-Coalescing Operators

Null-Forgiving Operator

Nullable Analysis Attributes
```

This topic covers:

- what `null` means;

- absence of value vs default value;
- `NullReferenceException`;
- nullable value types;
- `T?`;
- `Nullable<T>`;
- `HasValue`;
- `Value`;
- `GetValueOrDefault`;
- pattern matching nullable values;
- lifted operators;
- nullable Boolean values;
- nullable reference types;
- `string` vs `string?`;
- nullable contexts;
- `<Nullable>enable</Nullable>`;
- compiler null-state analysis;
- `not-null` vs `maybe-null`;
- `is null`;
- `is not null`;
- `== null`;
- `?.`;
- `?[]`;
- `??`;
- `??=`;
- C# 14 null-conditional assignment;
- null-forgiving `!`;
- `ArgumentNullException.ThrowIfNull`;
- nullability in parameters;
- nullability in return values;
- nullable DTO/domain design;
- nullable attributes;
- generics and nullability;
- database/API implications;
- common mistakes;
- performance considerations;
- security considerations;
- practical Fundraising Platform examples;
- exercises;
- interview questions.

The overall idea is:

EXAMPLE / FLOW

```
Value
    →
Can it legitimately be absent?
    →
No
    →
Use non-nullable type

Yes
    →
Use appropriate nullable type
    →
Check / propagate / substitute / reject null
```

✦ What Does null Mean?

Definition

`null` represents the **absence of an object reference or value**.

In plain language:

EXAMPLE / FLOW

```
null
=
"There is currently no value/object here."
```

For example:

C#

```
string? middleName = null;
```

This means:

☀ The person currently has no middle-name value available.

It does **not** mean:

EXAMPLE / FLOW

```
middleName = ""
```

and it does not mean:

EXAMPLE / FLOW

```
middleName = "null"
```

These are completely different values.

✦ null vs Empty String

Consider:

```C#
string? notes = null;
```

This means:

EXAMPLE / FLOW

```
No notes value exists.
```

But:

```C#
string notes = "";
```

means:

EXAMPLE / FLOW

```
A string exists,
but it contains zero characters.
```

And:

```C#
string notes = " ";
```

means:

EXAMPLE / FLOW

```
A string exists
and contains whitespace.
```

These states can have different business meanings.

✦ Fundraising Example

Suppose a gift has an optional note.

EXAMPLE / FLOW

```
NULL
    →
No note was supplied.

""
    →
A note value was supplied,
but it is empty.

"Thank you"
    →
Actual note exists.
```

Do not automatically treat all three states as equivalent unless the business rule says they are equivalent.

✦ null vs Zero

This difference is extremely important.

Suppose:

C#

```
decimal? matchingAmount = null;
```

could mean:

EXAMPLE / FLOW

```
Matching amount is unknown
or not applicable.
```

But:

C#

```
decimal matchingAmount = 0m;
```

means:

EXAMPLE / FLOW

```
The matching amount is known
and equals zero.
```

These are not always the same business state.

✦ Why Does null Exist?

Many real-world values can legitimately be missing.

Examples:

EXAMPLE / FLOW

```
Optional Campaign

Optional Middle Name

Optional Gift Notes

Optional Refund Date

Unknown Completion Date

Optional Parent Record

Optional API Response

Object Not Found
```

Without `null`, developers would sometimes need artificial values such as:

EXAMPLE / FLOW

```
-1

0

"UNKNOWN"

DateTime.MinValue
```

to represent absence.

Those sentinel values can create ambiguity.

Nullable types give us a clearer representation.

✦ NullReferenceException

One of the most common consequences of incorrect null handling is:

EXAMPLE / FLOW

```
NullReferenceException
```

Example:

C#

```
Gift? gift = null;

decimal amount =
    gift.TotalAmount;
```

There is no object behind:

EXAMPLE / FLOW

```
gift
```

so attempting to dereference it can fail at runtime.

Modern nullable reference types are primarily designed to help identify many of these situations **during compilation rather than after deployment**. Microsoft describes nullable reference types as compile-time annotations combined with null-state analysis; they don't create a new runtime reference type.

✦ What Does Dereference Mean?

Suppose:

C#

```
Gift? gift;
```

Using:

C#

```
gift.TotalAmount
```

means:

EXAMPLE / FLOW

```
Follow the reference
    →
Find Gift object
    →
Read TotalAmount
```

This is called:

EXAMPLE / FLOW

```
dereferencing
```

Other examples:

C#

```
gift.Process();
```

C#

```
gift.Campaign
```

C#

```
name.Length
```

```C#
values[0]
```

all require accessing something through a reference.

If that reference is `null`, the operation may fail.

✦ The Two Different Nullable Systems

This is extremely important.

C# has:

```EXAMPLE / FLOW
Nullable Value Types
```

and:

```EXAMPLE / FLOW
Nullable Reference Types
```

They use similar `?` syntax but they are technically different concepts.

✦ Nullable Value Types

Examples:

```C#
int?
decimal?
bool?
DateTime?
long?
```

These are actual nullable value-type representations based on:

```EXAMPLE / FLOW
Nullable<T>
```

✦ Nullable Reference Types

Examples:

```C#
string?
Gift?
Campaign?
Constituent?
```

These are **annotations describing developer intent and compiler null analysis**.

They are not different runtime CLR classes from:

```C#
string
Gift
Campaign
```

Microsoft explicitly notes that `string` and `string?`, for example, are the same runtime .NET type; the difference is compiler nullability analysis and annotations.

✦ Nullable Value Types

Why Value Types Need Special Null Support

You learned that types such as:

EXAMPLE / FLOW

```
int
long
decimal
bool
DateTime
Guid
```

are value types.

Normally:

```C#
int giftCount = null;
```

is invalid.

Likewise:

```C#
decimal giftAmount = null;
```

is invalid.

A normal non-nullable value type always has some value.

For example:

EXAMPLE / FLOW

```
default(int)
    →
0

default(bool)
    →
false

default(decimal)
    →
0
```

But sometimes the business value may genuinely be absent.

That is where nullable value types are used.

✦ Nullable Value Type Syntax

Append:

EXAMPLE / FLOW

```
?
```

to the value type.

Example:

C#

```
int? giftCount = null;
```

C#

```
decimal? refundedAmount = null;
```

C#

```
DateTime? refundedAt = null;
```

C#

```
bool? approvalDecision = null;
```

A nullable value type can contain:

EXAMPLE / FLOW

```
any normal T value
+
null
```

Microsoft defines `T?` for a value type as shorthand for `System.Nullable<T>`.

✦ T? and Nullable

These two declarations are equivalent:

```C#
int? giftCount = null;
```

and:

```C#
Nullable<int> giftCount =
    null;
```

Normally use:

```C#
int?
```

because it is shorter and idiomatic.

✦ Nullable Value Type Flow

Without nullable:

```EXAMPLE / FLOW
int
    →
Any valid int value
```

With nullable:

```EXAMPLE / FLOW
int?
    →
Any valid int value
    OR
    null
```

For Boolean:

```EXAMPLE / FLOW
bool
    →
true
false
```

Nullable Boolean:

EXAMPLE / FLOW

```
bool?
    →
true
false
null
```

✦ Fundraising Example — Optional Campaign ID

Suppose a gift may optionally be associated with a campaign.

A required identifier:

C#

```
long tenantId;
```

must always contain a value.

But an optional campaign identifier can be:

C#

```
long? campaignId;
```

Then:

C#

```
campaignId = 100;
```

means:

EXAMPLE / FLOW

```
Gift belongs to Campaign 100.
```

while:

C#

```
campaignId = null;
```

means:

EXAMPLE / FLOW

```
Gift is not currently associated
with a campaign.
```

✦ Nullable Date Example

Suppose a gift has not been refunded.

You could model:

```C#
DateTime? refundedAtUtc =
    null;
```

When refund happens:

```C#
refundedAtUtc =
    DateTime.UtcNow;
```

Flow:

```EXAMPLE / FLOW
Gift Created
    →
RefundedAtUtc = null

Later Refund
    →
RefundedAtUtc = actual timestamp
```

This communicates the state naturally.

✦ Nullable bool

A normal Boolean has:

```EXAMPLE / FLOW
true
false
```

A nullable Boolean has:

```EXAMPLE / FLOW
true
false
null
```

Example:

```C#
bool? approvalDecision =
    null;
```

Business interpretation might be:

EXAMPLE / FLOW

```
true
    →
Approved

false
    →
Rejected

null
    →
Not reviewed yet
```

This is useful only when the third state actually has business meaning.

✦ Do Not Use bool? Without Need

If your business rule has only:

EXAMPLE / FLOW

```
Yes
No
```

use:

C#

```
bool
```

not:

C#

```
bool?
```

Adding unnecessary nullable states creates more states your application must handle.

✦ HasValue

`Nullable<T>` exposes:

EXAMPLE / FLOW

```
HasValue
```

Example:

```C#
decimal? amount =
    1000m;

if (amount.HasValue)
{
    Console.WriteLine(
        "Amount exists.");
}
```

If:

```C#
amount = null;
```

then:

```EXAMPLE / FLOW
amount.HasValue
    →
false
```

If amount contains any decimal:

```EXAMPLE / FLOW
amount.HasValue
    →
true
```

✦ Value Property

A nullable value type also exposes:

```EXAMPLE / FLOW
Value
```

Example:

```C#
decimal? amount =
    1000m;

decimal actualAmount =
    amount.Value;
```

Result:

```EXAMPLE / FLOW
1000
```

But this is dangerous:

```C#
decimal? amount =
    null;

decimal actualAmount =
    amount.Value;
```

because `Value` cannot provide an underlying value when `HasValue` is false. The language specification states that accessing `Value` when a nullable value has no value throws `InvalidOperationException`.

✦ Safe HasValue + Value

```C#
decimal? amount =
    GetOptionalAmount();

if (amount.HasValue)
{
    Console.WriteLine(
        amount.Value);
}
```

Flow:

```EXAMPLE / FLOW
amount
    →
HasValue?
    →
No
    →
do not access Value

Yes
    →
Value is safe
```

✦ Modern Preferred Pattern Matching

Modern C# often reads more naturally using pattern matching:

```C#
decimal? amount =
    GetOptionalAmount();

if (amount is decimal actualAmount)
{
    Console.WriteLine(
        actualAmount);
}
```

This simultaneously:

EXAMPLE / FLOW

```
Checks value exists
+
extracts underlying value
```

Microsoft's current nullable-value guidance recommends this type-pattern style for checking and extracting nullable values.

✦ Pattern Matching Flow

EXAMPLE / FLOW

```
decimal? amount
    →
Contains decimal?
    →
Yes
    →
Store value in actualAmount

No
    →
Else branch
```

✦ GetValueOrDefault

Another option is:

C#

```
decimal? amount =
    null;

decimal result =
    amount.GetValueOrDefault();
```

Result:

EXAMPLE / FLOW

```
0
```

because:

EXAMPLE / FLOW

```
default(decimal)
=
0
```

You can also specify a fallback:

```C#
decimal result =
    amount.GetValueOrDefault(
        -1m);
```

Now if amount is null:

```EXAMPLE / FLOW
result = -1
```

Microsoft documents both overloads: one returns the underlying type's default value, while the other accepts a fallback value.

✦ GetValueOrDefault vs Business Meaning

Be careful.

Suppose:

```C#
decimal? refundedAmount =
    null;
```

Then:

```C#
decimal value =
    refundedAmount
        .GetValueOrDefault();
```

produces:

```EXAMPLE / FLOW
0
```

But you just converted:

```EXAMPLE / FLOW
No refund information
```

into:

```EXAMPLE / FLOW
Refund amount is exactly zero
```

Those may have different meanings.

Do not default nullable values merely to avoid handling null.

✦ ?? Is Often More Readable

Instead of:

```C#
decimal result =
    amount.GetValueOrDefault(
        0m);
```

you can often write:

```C#
decimal result =
    amount ?? 0m;
```

This reads:

☀ Use `amount` when it exists; otherwise use zero.

✦ Nullable Arithmetic

Nullable numeric operations often propagate `null`.

Example:

```C#
decimal? amount =
    100m;

decimal? fee =
    10m;

decimal? result =
    amount - fee;
```

Result:

EXAMPLE / FLOW

```
90
```

But:

```C#
decimal? amount =
    null;

decimal? fee =
    10m;

decimal? result =
    amount - fee;
```

Result:

EXAMPLE / FLOW

```
null
```

These are called **lifted operators**: operators from the underlying value type are lifted to nullable values, generally producing null when an operand is null.

✦ Nullable Arithmetic Flow

EXAMPLE / FLOW

```
Operand A
+
Operand B
    →
Are required values present?
    →
Yes
    →
Perform operation

No
    →
Result often null
```

✦ Example

C#

```
decimal? giftAmount =
    1000m;

decimal? processingFee =
    null;

decimal? netAmount =
    giftAmount
    - processingFee;
```

Result:

EXAMPLE / FLOW

```
null
```

C# cannot calculate a meaningful numeric answer because one required numeric value is absent.

✦ Supplying a Fallback Before Arithmetic

If your business rule says:

EXAMPLE / FLOW

```
Missing fee means zero fee
```

then explicitly write that rule:

C#

```
decimal netAmount =
    giftAmount
    - (processingFee ?? 0m);
```

This is clearer than allowing null to silently propagate.

✦ Nullable Comparison Operators

Nullable comparisons require care.

Suppose:

C#

```
int? value =
    null;
```

Then:

C#

```
bool result =
    value > 10;
```

produces:

EXAMPLE / FLOW

```
false
```

for relational comparisons when one operand is null.

Microsoft's nullable-value documentation specifies that relational operators such as `<`, `>`, `<=`, and `>=` return `false` when one or both nullable operands are null.

✦ Do Not Interpret false as Zero

Consider:

C#

```
decimal? amount =
    null;

bool largeGift =
    amount > 5000m;
```

Result:

EXAMPLE / FLOW

```
false
```

That does **not** mean:

EXAMPLE / FLOW

```
amount <= 5000
```

It could mean:

EXAMPLE / FLOW

```
amount has no value
```

If this distinction matters:

C#

```
if (amount is null)
{
    // Missing amount.
}
else if (amount >= 5000m)
{
    // Major gift.
}
```

✦ Nullable Equality

Equality handles null more naturally.

Example:

C#

```
int? first =
    null;

int? second =
    null;

bool equal =
    first == second;
```

Result:

EXAMPLE / FLOW

```
true
```

because both represent no value.

But:

```C#
int? first =
    10;

int? second =
    null;
```

gives:

```EXAMPLE / FLOW
false
```

✦ Nullable bool Logic

`bool?` can contain:

```EXAMPLE / FLOW
true
false
null
```

Some logical operators therefore follow three-valued logic.

For example, the behavior of:

```EXAMPLE / FLOW
&
|
```

with nullable Boolean values has special rules and doesn't always simply return null whenever one side is null. Microsoft explicitly calls out `bool?` as an exception to the ordinary lifted-operator null-propagation rule.

For normal business code, keep nullable Boolean logic explicit instead of writing difficult three-valued expressions.

✦ Better Nullable bool Handling

Instead of:

```C#
if (approvalDecision == true)
{
}
```

you can intentionally model all states:

```C#
if (approvalDecision is true)
{
    Console.WriteLine(
        "Approved");
}
else if (approvalDecision is false)
{
    Console.WriteLine(
        "Rejected");
}
else
{
    Console.WriteLine(
        "Pending review");
}
```

This makes all three business states visible.

✦ Nullable Reference Types

Now we move to one of the most important modern C# features.

Consider:

```C#
string donorName;
```

versus:

```C#
string? donorName;
```

In a nullable-aware project they communicate different intentions.

✦ Non-Nullable Reference Type

```C#
string donorName;
```

means:

☀ This reference is intended to contain a non-null string.

The compiler tries to help enforce that expectation.

✦ Nullable Reference Type

```C#
string? donorName;
```

means:

☀ This reference is intentionally allowed to be null.

Therefore callers must consider null before safely dereferencing it.

✦ Important Runtime Rule

This is extremely important:

EXAMPLE / FLOW

```
string
```

and:

EXAMPLE / FLOW

```
string?
```

are **not two different CLR classes**.

The `?` is primarily metadata/compiler intent used for nullable static analysis.

Microsoft states that nullable reference types are compile-time annotations and don't change the underlying runtime type.

✦ Why Nullable Reference Types Were Added

Older C# allowed:

C#

```
string name =
    null;
```

without expressing whether null was expected.

This created ambiguity.

Looking at:

C#

```
string FindDonor(long id)
```

a developer could not easily know:

C#

```
Does this always return string?

Can it return null?

Should I check null?

Is null a bug?
```

Modern nullable annotations make the contract clearer.

✦ Better API Contract

```C#
string? FindDonorName(
    long constituentId)
```

clearly communicates:

```EXAMPLE / FLOW
Donor name may not be found.
```

But:

```C#
string GetRequiredCurrencyCode()
```

communicates:

```EXAMPLE / FLOW
A currency code is expected.
```

✦ Enable Nullable Reference Types

Modern .NET projects normally use nullable analysis through the project setting:

```XML
<PropertyGroup>
    <Nullable>enable</Nullable>
</PropertyGroup>
```

Recent .NET templates enable nullable context by default, and the compiler supports the modes `enable`, `disable`, `warnings`, and `annotations`.

For modern application development, you should normally keep nullable analysis enabled.

✦ Nullable Context

With:

```XML
<Nullable>enable</Nullable>
```

the compiler distinguishes:

```C#
string
```

from:

```C#
string?
```

as part of static null analysis.

✦ File-Level Nullable Directive

C# also supports:

```C#
#nullable enable
```

and:

```C#
#nullable disable
```

plus:

```C#
#nullable restore
```

These can override project settings for a source region. Microsoft recommends project-level configuration as the normal end state, with directives especially useful during migration of legacy code.

For new code:

```EXAMPLE / FLOW
Prefer project-wide nullable enable.
```

✦ Null-State Analysis

Nullable reference types are not simply:

```EXAMPLE / FLOW
string?
```

syntax.

The compiler performs **flow-sensitive null-state analysis**.

It tracks each reference expression as approximately:

```EXAMPLE / FLOW
Not Null

or

Maybe Null
```

Microsoft calls these states `not-null` and `maybe-null`.

✦ Example

C#
```
string? donorName =
    GetDonorName();

Console.WriteLine(
    donorName.Length);
```

Compiler sees:

EXAMPLE / FLOW
```
donorName
    →
maybe-null
```

Therefore:

EXAMPLE / FLOW
```
donorName.Length
```

can produce a warning.

✦ After Null Check

C#
```
string? donorName =
    GetDonorName();

if (donorName is not null)
{
    Console.WriteLine(
        donorName.Length);
}
```

Inside the block, the compiler understands:

EXAMPLE / FLOW
```
donorName
    →
not-null
```

Therefore accessing:

C#
```
donorName.Length
```

is safe according to static analysis.

✦ Null-State Flow

EXAMPLE / FLOW

```
string? donorName
    →
maybe-null

Check:
donorName is not null
    →
true branch
    →
not-null

false branch
    →
null
```

This is called:

EXAMPLE / FLOW

```
flow analysis
```

because the compiler changes its understanding based on program flow.

✦ Nullable Warning Is Not an Error

Consider:

C#

```
string? name =
    null;

Console.WriteLine(
    name.Length);
```

With normal nullable analysis, this typically generates a compiler warning.

It may still compile depending on project warning settings.

Why?

Nullable reference types are primarily a static-analysis safety feature.

The runtime still allows reference variables to contain null.

This is why you should:

EXAMPLE / FLOW

```
Treat nullable warnings seriously.
```

Many teams configure important warnings as build failures.

✦ Non-Nullable Does Not Guarantee Runtime Non-Null

This is another important interview point.

Suppose:

```
C#
string name;
```

is declared non-nullable.

That is a **contract/intention**.

It does not magically make null impossible at the CLR level.

Null could still appear through:

```
EXAMPLE / FLOW
Legacy libraries

Reflection

Deserialization

Incorrect null-forgiving use

Interop

Bad initialization

External frameworks
```

So nullable reference types improve safety but are not runtime null-proofing.

✦ Initialization Requirement

In nullable-aware code:

```
C#
public class Constituent
{
    public string DisplayName
    {
        get;
        set;
    }
}
```

the compiler expects `DisplayName` to receive a non-null value during initialization.

Otherwise, it can warn that the non-nullable property is not initialized.

Solutions include:

EXAMPLE / FLOW

```
Constructor initialization

required members

initializers

nullable type when null is legitimate
```

Do not simply add:

C#

```
= null!;
```

everywhere unless framework behavior truly guarantees initialization later.

✦ Constructor Initialization

Good:

C#

```
public sealed class Campaign
{
    public Campaign(
        string name)
    {
        Name =
            name;
    }

    public string Name
    {
        get;
    }
}
```

The constructor guarantees:

EXAMPLE / FLOW

```
Name receives value
```

before a usable object exists.

✦ required Preview

Modern C# also provides:

C#

```
required
```

for members that must be initialized by object construction/initialization.

Example:

```C#
public sealed class Campaign
{
    public required string Name
    {
        get;
        init;
    }
}
```

Then:

```C#
var campaign =
    new Campaign
    {
        Name =
            "Annual Giving"
    };
```

We will study `required`, constructors, and object initialization properly in the OOP topics.

✦ is null

The pattern:

```C#
value is null
```

checks whether a value is null.

Example:

```C#
if (gift is null)
{
    return;
}
```

This is clean modern C#.

✦ is not null

```C#
if (gift is not null)
{
    ProcessGift(
        gift);
}
```

This performs the opposite test.

Microsoft recommends these null patterns as part of modern null-safe code.

✦ == null

You can also write:

```C#
if (gift == null)
{
}
```

For many ordinary types this works perfectly.

However:

```EXAMPLE / FLOW
==
```

can be overloaded by custom types.

Pattern matching:

```C#
gift is null
```

does not depend on overloaded equality semantics.

Therefore `is null` / `is not null` are often especially clear for pure null checks.

✦ Guard Clause Example

```C#
static void ProcessGift(
    Gift? gift)
{
    if (gift is null)
    {
        throw new ArgumentNullException(
            nameof(gift));
    }

    Console.WriteLine(
        gift.TotalAmount);
}
```

After the guard:

```EXAMPLE / FLOW
gift
    →
known non-null
```

✦ ArgumentNullException.ThrowIfNull

Modern .NET provides:

```C#
ArgumentNullException.ThrowIfNull(
    gift);
```

instead of manually writing:

```C#
if (gift is null)
{
    throw new ArgumentNullException(
        nameof(gift));
}
```

Example:

```C#
static void ProcessGift(
    Gift gift)
{
    ArgumentNullException.ThrowIfNull(
        gift);

    // Continue processing.
}
```

Use this when:

```EXAMPLE / FLOW
null means caller violated method contract
```

rather than when null is an expected valid result.

✦ Expected null vs Invalid null

This distinction is essential.

Expected null

```C#
Gift? FindGift(
    long giftId)
```

A gift may not exist.

Returning null can be intentional.

Invalid null

```C#
void ProcessGift(
    Gift gift)
```

This method requires a `Gift`.

Passing null violates the contract.

Then throwing:

EXAMPLE / FLOW

```
ArgumentNullException
```

may be appropriate.

✦ Do Not Throw for Every Nullable Value

Suppose:

C#

```
Campaign? campaign =
    gift.Campaign;
```

If campaign is legitimately optional, null is not exceptional.

Do not write:

C#

```
throw new Exception(
    "Campaign null!");
```

just because a reference is null.

Null handling should match the domain meaning.

✦ Null-Conditional Operator ?.

Definition

`?.` accesses a member only when the object on the left is non-null.

Example:

C#

```
string? campaignName =
    gift.Campaign?.Name;
```

If:

EXAMPLE / FLOW

```
gift.Campaign exists
```

then:

EXAMPLE / FLOW

```
Name is accessed
```

If:

EXAMPLE / FLOW

```
gift.Campaign is null
```

the expression returns:

EXAMPLE / FLOW

```
null
```

rather than dereferencing the missing campaign.

Microsoft specifies that null-conditional operators short-circuit: once a null is encountered, later conditional accesses in the chain are skipped.

✦ ?. Flow

EXAMPLE / FLOW

```
Campaign
    →
null?
    →
Yes
    →
Return null

No
    →
Read Name
```

✦ Null-Conditional Chain

Example:

C#

```
string? city =
    gift
        ?.Constituent
        ?.PrimaryAddress
        ?.City;
```

Flow:

EXAMPLE / FLOW

```
Gift exists?
    →
Constituent exists?
    →
Primary Address exists?
    →
City

Any null?
    →
Stop chain
    →
Return null
```

✦ Important Parentheses Rule

This:

C#

```
gift?.Campaign?.Name
```

maintains conditional chaining.

But introducing parentheses can terminate a null-conditional chain in some expressions.

Microsoft specifically warns that parentheses such as:

EXAMPLE / FLOW

```
(A?.B).C()
```

can change the short-circuit behavior.

Keep null chains simple and readable.

✦ ?[] Null-Conditional Indexing

Suppose:

C#

```
string[]? codes =
    null;
```

Instead of:

C#

```
string first =
    codes[0];
```

you can use:

```C#
string? first =
    codes?[0];
```

If the array is null:

EXAMPLE / FLOW

```
first = null
```

✦ ?[] Does Not Protect Invalid Indexes

Suppose:

```C#
string[] codes =
{
    "A"
};
```

Then:

```C#
string? value =
    codes?[5];
```

still fails because:

EXAMPLE / FLOW

```
codes is not null
```

and index:

EXAMPLE / FLOW

```
5
```

is invalid.

`?[]` protects against:

EXAMPLE / FLOW

```
null collection
```

not:

EXAMPLE / FLOW

```
invalid index
```

Microsoft explicitly documents this distinction.

✦ ?. with Value-Type Result

Suppose:

```
C#
string? name =
    null;
```

`Length` normally returns:

```
EXAMPLE / FLOW
int
```

But:

```
C#
int? length =
    name?.Length;
```

returns:

```
EXAMPLE / FLOW
int?
```

Why?

If `name` is null, there is no integer length result.

Therefore C# lifts the result into:

```
EXAMPLE / FLOW
int?
```

Microsoft documents this behavior for null-conditional access to non-nullable value-type members.

✦ Null-Coalescing Operator ??

Definition

The null-coalescing operator:

```
EXAMPLE / FLOW
??
```

returns its left value if non-null.

Otherwise it evaluates and returns its right value.

Example:

```C#
string? donorName =
    null;

string displayName =
    donorName
    ??
    "Anonymous Donor";
```

Result:

```EXAMPLE / FLOW
Anonymous Donor
```

✦ ?? Flow

```EXAMPLE / FLOW
Left side
    →
null?
    →
No
    →
Use left side

Yes
    →
Evaluate right side
    →
Use right side
```

The right-hand expression is not evaluated when the left side is already non-null.

✦ Fundraising Example

```C#
string displayNotes =
    gift.Notes
    ??
    "No notes";
```

But ask whether:

```EXAMPLE / FLOW
null
```

should really mean:

EXAMPLE / FLOW

```
"No notes"
```

before applying the fallback.

✦ Chaining ??

C#

```
string displayName =
    preferredName
    ??
    legalName
    ??
    "Anonymous";
```

Meaning:

EXAMPLE / FLOW

```
Preferred name available?
    →
Use it

Otherwise legal name available?
    →
Use it

Otherwise
    →
Anonymous
```

`??` is right-associative.

✦ Combining ?. and ??

One of the most useful patterns:

C#

```
string campaignName =
    gift.Campaign?.Name
    ??
    "No Campaign";
```

Flow:

EXAMPLE / FLOW

```
Campaign exists?
    →
No
    →
"No Campaign"

Yes
    →
Name exists?
    →
Use Name

Otherwise
    →
"No Campaign"
```

✦ Null-Coalescing Assignment ??=

`??=` assigns the right-hand value only when the left-hand variable is null.

Example:

C#

```
List<long>? giftIds =
    null;

giftIds ??=
    new List<long>();
```

Now:

EXAMPLE / FLOW

```
giftIds
```

contains a list.

Call again:

C#

```
giftIds ??=
    LoadGiftIds();
```

If `giftIds` is already non-null:

EXAMPLE / FLOW

```
LoadGiftIds()
```

is not called.

Microsoft documents `??=` as lazy assignment whose right-hand expression is skipped when the left side is already non-null.

✦ Lazy Initialization Example

C#

```
private List<Gift>?
    _pendingGifts;

private List<Gift>
    PendingGifts
    =>
    _pendingGifts
    ??=
    LoadPendingGifts();
```

First access:

EXAMPLE / FLOW

```
_pendingGifts = null
    →
LoadPendingGifts()
    →
cache result
```

Later access:

EXAMPLE / FLOW

```
_pendingGifts already exists
    →
reuse existing list
```

Thread safety is a separate concern.

✦ C# 14 Null-Conditional Assignment

C# 14 allows null-conditional access as an assignment target in eligible cases.

Example:

C#

```
Campaign? campaign =
    GetCampaign();

campaign?.Description =
    "Updated campaign";
```

Meaning:

EXAMPLE / FLOW

```
campaign exists?
    →
Yes
    →
perform assignment

No
    →
do nothing
```

The right-hand side is evaluated only when the target object exists. This is a C# 14 language feature.

✦ When Null-Conditional Assignment Is Appropriate

Good:

C#

```
campaign?.LastViewedAtUtc =
    DateTime.UtcNow;
```

when:

EXAMPLE / FLOW

```
No campaign
=
No update required
```

Potentially bad:

C#

```
requiredGift?.Process();
```

if `requiredGift == null` represents a serious business/programming error.

Do not use `?.` merely to hide broken state.

✦ Null-Forgiving Operator !

Definition

Postfix:

EXAMPLE / FLOW

```
!
```

is called the:

EXAMPLE / FLOW

```
null-forgiving operator
```

Example:

C#

```
string? donorName =
    GetName();

int length =
    donorName!.Length;
```

The `!` tells the **compiler**:

☀ Treat this expression as non-null for nullable analysis.

It does **not** perform a runtime null check and has no runtime protective behavior.

✦ Important ! Flow

EXAMPLE / FLOW

```
Compiler sees:
maybe-null

Developer writes:
!

Compiler now assumes:
not-null

Runtime value:
unchanged
```

If value is actually:

EXAMPLE / FLOW

```
null
```

then:

C#

```
donorName!.Length
```

can still throw.

✦ Bad Use of !

```C#
Gift gift =
    FindGift(id)!;

ProcessGift(
    gift);
```

If `FindGift` really can return null, you've merely hidden a compiler warning.

You have not fixed the business logic.

✦ Better

```C#
Gift? gift =
    FindGift(id);

if (gift is null)
{
    return;
}

ProcessGift(
    gift);
```

or perhaps:

```C#
Gift gift =
    FindGift(id)
    ??
    throw new GiftNotFoundException(
        id);
```

depending on the domain behavior.

✦ When ! Can Be Appropriate

The null-forgiving operator is useful when:

EXAMPLE / FLOW

```
Compiler cannot prove safety
but
developer/framework contract genuinely guarantees safety.
```

Examples may include:

EXAMPLE / FLOW

```
Certain framework initialization patterns

Tests deliberately passing null

Reflection/framework-populated properties

Complex invariants compiler cannot understand
```

Use it intentionally and sparingly.

✦ Never Use ! as a Cleanup Tool

Do not resolve every warning with:

```C#
!
```

Example:

```C#
customer!.Address!.City!.Length
```

This effectively disables the compiler's help at every level.

Investigate why those members are nullable.

✦ Null Check + Throw Expression

A concise pattern:

```C#
Gift gift =
    FindGift(id)
    ??
    throw new InvalidOperationException(
        "Gift not found.");
```

Flow:

```C#
FindGift
    →
Gift returned?
    →
Yes
    →
assign Gift

No
    →
throw exception
```

This is useful when null cannot be tolerated beyond that point.

✦ Parameters and Nullability

Compare:

```C#
void ProcessGift(
    Gift gift)
```

with:

```C#
void ProcessGift(
    Gift? gift)
```

They describe different contracts.

✦ Non-Nullable Parameter

```C#
void ProcessGift(
    Gift gift)
```

communicates:

EXAMPLE / FLOW

```
Caller is expected to provide a Gift.
```

The method may still validate runtime input if crossing an unsafe boundary.

✦ Nullable Parameter

```C#
void ProcessGift(
    Gift? gift)
```

communicates:

EXAMPLE / FLOW

```
null is a legitimate supported input.
```

Therefore the method itself should explicitly handle that state.

✦ Do Not Add ? Just to Remove Warnings

Bad approach:

```C#
void ProcessGift(
    Gift? gift)
```

when the real rule is:

```EXAMPLE / FLOW
Gift is required.
```

You have weakened the API contract.

Fix the initialization/caller logic instead.

✦ Return Values and Nullability

Compare:

```C#
Gift GetGift(
    long giftId)
```

with:

```C#
Gift? FindGift(
    long giftId)
```

The naming and type can communicate behavior.

✦ Required Return

```C#
Gift GetRequiredGift(
    long giftId)
```

might:

```EXAMPLE / FLOW
Return Gift
or
throw when not found
```

✦ Optional Return

```C#
Gift? FindGift(
    long giftId)
```

might:

EXAMPLE / FLOW

```
Return Gift
or
null when absent
```

Both are valid APIs depending on expected behavior.

✦ Good API Design Principle

Use nullable annotations to communicate reality.

If null can legitimately occur:

EXAMPLE / FLOW

```
annotate it
```

If null should never occur:

EXAMPLE / FLOW

```
do not mark it nullable merely for convenience
```

Nullability becomes part of your API contract.

✦ Fundraising Service Example

C#

```
public Gift? FindGift(
    long giftId)
{
    // Return Gift or null.
}
```

Caller:

C#

```
Gift? gift =
    giftService.FindGift(
        giftId);

if (gift is null)
{
    Console.WriteLine(
        "Gift not found.");

    return;
}

Console.WriteLine(
    gift.TotalAmount);
```

Flow:

EXAMPLE / FLOW

```
Find Gift
    →
Found?
    →
Yes
    →
Process

No
    →
Handle absence
```

✦ Nullability and DTO Design

Suppose we create:

C#

```
public sealed class CreateGiftRequest
{
    public long ConstituentId
    {
        get;
        init;
    }

    public decimal TotalAmount
    {
        get;
        init;
    }

    public long? CampaignId
    {
        get;
        init;
    }

    public string? Notes
    {
        get;
        init;
    }
}
```

This communicates:

EXAMPLE / FLOW

```
ConstituentId
    →
required numeric value

TotalAmount
    →
required numeric value

CampaignId
    →
optional

Notes
    →
optional
```

That is much clearer than making every property nullable.

✦ Nullable Annotation Is Not Full Validation

This is very important for ASP.NET Core.

Suppose:

C#

```
public string DonorName
{
    get;
    init;
}
```

The non-nullable annotation says:

EXAMPLE / FLOW

```
This property is intended to be non-null.
```

It does **not** prove:

EXAMPLE / FLOW

```
Name isn't empty

Name isn't whitespace

Name has valid length

Name contains allowed characters
```

Nullability is only one part of validation.

✦ Type Rule vs Null Rule vs Business Rule

Example:

```C#
decimal TotalAmount
```

Type rule:

```EXAMPLE / FLOW
Must be decimal.
```

Null rule:

```EXAMPLE / FLOW
Cannot represent null.
```

Business rule:

```EXAMPLE / FLOW
Must be greater than zero.
```

All three are separate concerns.

✦ Null vs Empty in APIs

Suppose an incoming DTO has:

```C#
string? Notes
```

You must decide whether these are equivalent:

```JSON
{
  "notes": null
}
```

```JSON
{
  "notes": ""
}
```

```JSON
{
}
```

depending on API/deserialization/business semantics.

Do not assume they always mean the same thing.

✦ Nullability and Database Models

Database nullable columns often naturally map to nullable CLR types.

For example:

EXAMPLE / FLOW

```
SQL nullable BIGINT
    →
long?

SQL nullable DATE/DATETIME
    →
DateTime? / DateOnly?

SQL nullable text
    →
string?
```

But the application domain should still determine whether null is truly valid.

✦ EF Core Preview

Nullable reference annotations can influence how EF Core interprets model optionality. Microsoft explicitly notes that EF Core reads nullable metadata and can interpret nullable reference members as optional and non-nullable members as required.

We will study this properly in the EF Core roadmap topics.

For now:

EXAMPLE / FLOW

```
Nullability annotations
can affect more than compiler warnings
when frameworks inspect metadata.
```

✦ Null-State Analysis and Method Calls

Suppose:

C#

```
string? name =
    GetName();

if (name is null)
{
    return;
}

Console.WriteLine(
    name.Length);
```

Compiler sees:

```
C#
after return guard
    →
name cannot be null
```

This is another reason guard clauses work nicely with nullable analysis.

✦ Flow Analysis with Assignment

```
C#
string? name =
    null;

name =
    "Jayanth";

Console.WriteLine(
    name.Length);
```

Although the variable's declared type is:

```
EXAMPLE / FLOW
string?
```

at this point the compiler knows its current state is:

```
EXAMPLE / FLOW
not-null
```

Declared nullability and current null-state are related but not identical.

✦ Declared Type vs Current Null-State

Example:

```
C#
string? name =
    GetName();
```

Declared annotation:

```
EXAMPLE / FLOW
nullable
```

Current compiler state might be:

```
EXAMPLE / FLOW
maybe-null
```

After:

```C#
if (name is null)
{
    return;
}
```

declared type remains:

```EXAMPLE / FLOW
string?
```

but current flow state becomes:

```EXAMPLE / FLOW
not-null
```

This is an advanced but essential distinction.

✦ Null-State Analysis Is Flow Sensitive

Example:

```C#
string? name =
    GetName();

if (name is not null)
{
    Console.WriteLine(
        name.Length);
}

Console.WriteLine(
    name.Length);
```

Inside the `if`:

```EXAMPLE / FLOW
name
    →
not-null
```

After the block:

```EXAMPLE / FLOW
name
    →
may still be null
```

because the false path continues.

So compiler may warn on the second dereference.

✦ Better Guard Clause

```C#
string? name =
    GetName();

if (name is null)
{
    return;
}

Console.WriteLine(
    name.Length);
```

After the guard:

EXAMPLE / FLOW

```
all continuing paths
    →
name is non-null
```

✦ Null Analysis Across Custom Methods

Suppose you create:

```C#
static bool HasValue(
    string? value)
{
    return value is not null;
}
```

Then:

```C#
if (HasValue(name))
{
    Console.WriteLine(
        name.Length);
}
```

Depending on the method contract, the compiler may not automatically understand that:

EXAMPLE / FLOW

```
HasValue(name) == true
```

guarantees `name` isn't null.

This is where nullable analysis attributes become useful.

✦ Nullable Analysis Attributes

Advanced APIs can communicate richer nullability contracts using:

EXAMPLE / FLOW

```
System.Diagnostics.CodeAnalysis
```

attributes.

Important attributes include:

EXAMPLE / FLOW

```
AllowNull

DisallowNull

MaybeNull

NotNull

MaybeNullWhen

NotNullWhen

NotNullIfNotNull

MemberNotNull

MemberNotNullWhen

DoesNotReturn

DoesNotReturnIf
```

Microsoft provides these specifically to improve compiler null-state analysis for contracts that simple `?` annotations cannot fully express.

✦ NotNullWhen

Suppose:

C#

```
static bool HasValue(
    [NotNullWhen(true)]
    string? value)
{
    return value is not null;
}
```

Now the attribute tells the compiler:

C#

```
If method returns true,
value is definitely non-null.
```

Caller:

```C#
if (HasValue(name))
{
    Console.WriteLine(
        name.Length);
}
```

The compiler can understand the contract more accurately.

✦ Try Pattern Example

A common pattern:

```C#
bool TryFindGift(
    long id,
    [NotNullWhen(true)]
    out Gift? gift)
```

Contract:

```C#
returns true
    →
gift is non-null

returns false
    →
gift may be null
```

Caller:

```C#
if (TryFindGift(
        giftId,
        out Gift? gift))
{
    Console.WriteLine(
        gift.TotalAmount);
}
```

The compiler can know `gift` is non-null in the success branch.

✦ MaybeNull

`[MaybeNull]` can describe an API where a value normally appears non-nullable in its type declaration but the implementation may return null.

This is especially useful in certain generic APIs.

For ordinary application methods, prefer:

```C#
Gift?
```

when possible because that contract is simpler.

✦ AllowNull

`[AllowNull]` says:

EXAMPLE / FLOW

```
The declared type is generally non-null,
but callers are allowed to provide null.
```

This is useful for specialized setter/API contracts.

It is not something most everyday business methods need.

✦ DisallowNull

`[DisallowNull]` describes the reverse kind of precondition:

EXAMPLE / FLOW

```
The member may be nullable when read,
but callers shouldn't explicitly assign null.
```

Again, this is more common in library/API contract design than ordinary business service code.

✦ MemberNotNull

Suppose a helper method initializes a field:

C#

```
private string? _token;

private void Initialize()
{
    _token =
        LoadToken();
}
```

If the compiler can't infer that `_token` is guaranteed non-null after the method, advanced library code can use:

EXAMPLE / FLOW

```
MemberNotNull
```

to describe that postcondition.

Use these attributes to make contracts accurate, not merely to silence warnings.

✦ Nullability and Generics — Preview

Generics make `T?` more complicated because:

EXAMPLE / FLOW

```
T
```

could represent:

EXAMPLE / FLOW

```
value type
or
reference type
```

For example:

C#

```
T? Find<T>()
```

does not always mean exactly the same thing as:

C#

```
int?
```

Microsoft's current nullable reference documentation explains that generic constraints affect how `T?` is interpreted.

✦ Generic Constraints Preview

Important later:

C#

```
where T : class
```

means a non-nullable reference type constraint.

C#

```
where T : class?
```

allows nullable or non-nullable reference types.

C#

```
where T : struct
```

requires a non-nullable value type, and `T?` means `Nullable<T>`.

C#

```
where T : notnull
```

requires a non-nullable value or reference type.

We will revisit these in the Generics topic.

✦ default and Nullability

For reference types:

```C#
string? value =
    default;
```

produces:

```
EXAMPLE / FLOW
null
```

For:

```C#
string value =
    default;
```

the runtime default is still null, but nullable analysis warns because the declared contract says `value` should not be null.

This demonstrates again:

```
EXAMPLE / FLOW
Non-nullable reference annotation
≠
new runtime representation
```

✦ default! Pattern

You may encounter framework/library code such as:

```C#
public string Name
{
    get;
    set;
} = default!;
```

This tells the compiler:

```
EXAMPLE / FLOW
Trust me;
something will initialize this later.
```

This pattern appears in framework-managed models, but do not use it casually.

If you control construction, prefer real initialization.

✦ Null Object Pattern — Design Alternative

Sometimes repeated null checks can be avoided by providing a real object representing “nothing.”

Example concept:

EXAMPLE / FLOW

```
INotificationService
    →
EmailNotificationService

or

NoOpNotificationService
```

Instead of:

C#

```
notificationService?.Send();
```

you always have an implementation:

C#

```
notificationService.Send();
```

where the no-op implementation does nothing.

This is called the:

EXAMPLE / FLOW

```
Null Object Pattern
```

It can be useful when:

EXAMPLE / FLOW

```
absence represents behavior
rather than missing data
```

Do not use it for ordinary missing database values.

✦ Nullability and LINQ Preview

Methods such as:

EXAMPLE / FLOW

```
FirstOrDefault

SingleOrDefault
```

can produce default values when nothing is found.

For reference types, default is often:

EXAMPLE / FLOW

```
null
```

Example:

C#

```
Gift? gift =
    gifts.FirstOrDefault(
        g => g.GiftId == giftId);
```

Therefore nullability becomes extremely important when using LINQ.

We will study this deeply in the LINQ topic.

✦ Nullability and Dictionary Preview

A lookup might fail.

Instead of:

C#

```
Gift gift =
    dictionary[id];
```

you may later use:

C#

```
if (dictionary.TryGetValue(
        id,
        out Gift? gift))
{
}
```

The `Try...` pattern communicates success separately from the returned value.

Collections and nullable contracts work together heavily.

✦ Nullability and Async Preview

Suppose:

C#

```
Task<Gift?>
    FindGiftAsync(
        long giftId)
```

means:

```
C#
Asynchronous operation
    →
eventually returns
Gift or null
```

This is different from:

```
C#
Task<Gift>
```

which promises a non-null `Gift` according to its API contract.

Later async and Web API topics will rely on this.

✦ Null Is Not Always an Error

Examples where null may be perfectly valid:

```
EXAMPLE / FLOW
Optional Campaign

Optional Notes

Not Yet Completed Timestamp

Parent Does Not Exist

Lookup Did Not Find Object
```

Do not throw exceptions simply because null exists.

✦ Null Can Also Represent a Bug

Examples:

```
EXAMPLE / FLOW
Required Configuration Missing

Required Service Dependency Missing

Required Gift Missing After Invariant Says It Exists

Required Authentication Context Missing

Required Currency Missing
```

The correct response depends on the contract.

✦ Business Modeling Rule

Ask:

EXAMPLE / FLOW

```
What does null mean here?
```

If you cannot answer clearly, your model may need improvement.

Good nullable design makes absence meaningful.

✦ Practical Fundraising Model

C#

```
public sealed class Gift
{
    public long GiftId
    {
        get;
        init;
    }

    public long ConstituentId
    {
        get;
        init;
    }

    public long? CampaignId
    {
        get;
        init;
    }

    public decimal TotalAmount
    {
        get;
        init;
    }

    public string CurrencyCode
    {
        get;
        init;
    } = "USD";

    public string? Notes
    {
        get;
        init;
    }

    public DateTime? RefundedAtUtc
    {
        get;
        init;
    }
}
```

Interpretation:

EXAMPLE / FLOW

```
GiftId
    →
required

ConstituentId
    →
required

CampaignId
    →
optional

TotalAmount
    →
required

CurrencyCode
    →
required

Notes
    →
optional

RefundedAtUtc
    →
optional until refund occurs
```

This model communicates business intent through types.

✦ Example — Display Optional Campaign

C#

```
string campaignDisplay =
    gift.Campaign?.Name
    ??
    "No Campaign";
```

✦ Example — Optional Refund Date

C#

```
if (gift.RefundedAtUtc
    is DateTime refundedAt)
{
    Console.WriteLine(
        $"Refunded at {refundedAt:u}");
}
else
{
    Console.WriteLine(
        "Gift has not been refunded.");
}
```

✦ Example — Required Gift Lookup

Suppose application logic requires a gift:

C#

```
Gift gift =
    repository.FindGift(
        giftId)
    ??
    throw new InvalidOperationException(
        $"Gift {giftId} was not found.");
```

After this expression:

EXAMPLE / FLOW

```
gift
    →
known non-null
```

✦ Example — Optional Gift Lookup

If not finding a gift is normal:

C#

```
Gift? gift =
    repository.FindGift(
        giftId);

if (gift is null)
{
    return null;
}
```

Do not throw if the application's contract expects absence.

✦ Common Mistake — Using ! Everywhere

Bad:

```C#
string city =
    gift!
        .Constituent!
        .Address!
        .City!;
```

This does not make your code safe.

It merely tells the compiler not to warn you.

Correct the model or handle the missing states.

✦ Common Mistake — Making Everything Nullable

Example:

```C#
public long? GiftId { get; set; }

public decimal? Amount { get; set; }

public string? Currency { get; set; }

public bool? IsActive { get; set; }
```

when none of those values is legitimately optional.

This weakens your contracts and forces unnecessary null handling everywhere.

Use nullable types only when absence is meaningful.

✦ Common Mistake — Making Nothing Nullable

The opposite problem:

```C#
public string Notes { get; set; }
```

when notes are legitimately optional.

Then developers may:

EXAMPLE / FLOW

```
lie to compiler

use null!

store empty strings

add fake defaults
```

instead of accurately modeling the domain.

✦ Common Mistake — Null → Zero Without Thinking

C#

```
decimal amount =
    optionalAmount
    ??
    0m;
```

Ask:

EXAMPLE / FLOW

```
Does missing amount actually mean zero?
```

If not, this conversion loses information.

✦ Common Mistake — Null → Empty String Automatically

C#

```
string notes =
    gift.Notes
    ??
    "";
```

This may collapse two distinct states:

EXAMPLE / FLOW

```
No note
```

and:

EXAMPLE / FLOW

```
Empty note
```

Only do this when the business/UI requirement intentionally treats them the same.

✦ Common Mistake — Accessing Nullable.Value Directly

Risky:

```C#
DateTime date =
    gift.RefundedAtUtc.Value;
```

unless you have already proved:

```EXAMPLE / FLOW
HasValue = true
```

Prefer:

```C#
if (gift.RefundedAtUtc
    is DateTime date)
{
}
```

or an appropriate fallback.

✦ Common Mistake — Using ?. to Hide Required State

Bad:

```C#
requiredPayment?.Process();
```

If payment must exist, silently doing nothing hides a serious error.

Better:

```C#
ArgumentNullException.ThrowIfNull(
    requiredPayment);

requiredPayment.Process();
```

or fix object construction so invalid state cannot exist.

✦ Common Mistake — Non-Nullable Means Runtime Guarantee

Wrong:

```EXAMPLE / FLOW
string
=
cannot ever be null
```

Correct:

EXAMPLE / FLOW

```
string
=
developer/compiler contract
that value is intended to be non-null
```

Runtime null is still possible through unsafe boundaries or broken contracts.

✦ Common Mistake — Ignoring Nullable Warnings

A warning such as:

EXAMPLE / FLOW

```
Dereference of a possibly null reference
```

should not automatically be suppressed.

Ask:

EXAMPLE / FLOW

```
Why does compiler think this can be null?
```

Often it reveals a genuine edge case.

✦ Common Mistake — Unnecessary Nullable bool

Avoid:

C#

```
bool? isActive;
```

if:

EXAMPLE / FLOW

```
Active
Inactive
```

are the only legitimate states.

Use nullable Boolean only when a third state has meaning.

✦ Common Mistake — Confusing Nullable Value and Nullable Reference ?

These look similar:

C#

```
int?
```

```C#
string?
```

but they are not implemented the same way.

`int?`:

```EXAMPLE / FLOW
System.Nullable<int>
```

`string?`:

```EXAMPLE / FLOW
same runtime string reference type
+
compiler nullability annotation
```

This is one of the most important interview distinctions.

✦ Performance Considerations

Null checks themselves are normally extremely cheap.

Example:

```C#
if (gift is null)
{
}
```

is not a performance concern in normal business code.

Do not remove useful null validation for micro-optimization.

✦ Nullable Overhead

A nullable value type must represent:

```EXAMPLE / FLOW
HasValue
+
underlying value
```

so it has additional representation compared with the underlying value alone.

For normal business applications, this is rarely a reason to avoid correct nullable modeling.

Correctness comes first.

✦ Avoid Unnecessary Boxing

From the previous topic, remember that boxing nullable value types has special behavior:

EXAMPLE / FLOW

```
nullable contains value
    →
underlying value is boxed

nullable is null
    →
resulting object reference is null
```

Do not force nullable values through `object` unnecessarily in hot paths.

✦ ?. Does Not Magically Improve Performance

This:

C#

```
gift?.Campaign?.Name
```

is primarily about:

EXAMPLE / FLOW

```
safe concise semantics
```

not performance optimization.

Use it because it expresses optional navigation clearly.

✦ Avoid Repeated Expensive Fallbacks

This is safe because `??` short-circuits:

C#

```
Gift gift =
    cachedGift
    ??
    LoadGiftFromDatabase();
```

`LoadGiftFromDatabase()` runs only when `cachedGift` is null.

Still, ensure that calling a database query as an expression is readable and appropriate to your architecture.

✦ Concurrency Consideration

Suppose a property can change from another thread.

This:

```C#
if (service.CurrentGift is not null)
{
    Process(
        service.CurrentGift);
}
```

may read the property more than once.

A safer design can sometimes capture it:

```C#
Gift? gift =
    service.CurrentGift;

if (gift is not null)
{
    Process(
        gift);
}
```

or use APIs designed for thread-safe access.

Do not assume nullability analysis itself provides thread synchronization.

✦ Security Considerations

Null handling can affect security.

Suppose:

```C#
User? user =
    GetAuthenticatedUser();
```

This would be dangerous:

```C#
string role =
    user?.Role
    ??
    "Admin";
```

You just turned:

EXAMPLE / FLOW

```
Missing user
```

into:

EXAMPLE / FLOW

```
Admin
```

The fallback is security-sensitive.

✦ Fail Closed

Security logic should normally fail safely.

Example:

```C#
bool isAdministrator =
    user?.IsAdministrator
    ??
    false;
```

If user is missing:

EXAMPLE / FLOW

```
false
```

is generally safer than granting access.

✦ Missing Security Context Should Often Be Explicit

Even:

```C#
?? false
```

may hide a serious authentication bug.

For critical state, it can be better to reject immediately:

```C#
if (user is null)
{
    throw new UnauthorizedAccessException();
}
```

The correct behavior depends on the system boundary.

✦ Practical Exercise — Nullable Value

```C#
decimal? amount =
    null;
```

What is:

```C#
amount.HasValue
```

Answer:

EXAMPLE / FLOW

```
false
```

✦ Practical Exercise — Nullable Value Extraction

C#

```
decimal? amount =
    1000m;

decimal value =
    amount.Value;
```

Result:

EXAMPLE / FLOW

```
1000
```

✦ Practical Exercise — Pattern Match

C#

```
decimal? amount =
    1000m;

if (amount is decimal value)
{
    Console.WriteLine(
        value);
}
```

Output:

EXAMPLE / FLOW

```
1000
```

✦ Practical Exercise — Fallback

C#

```
decimal? fee =
    null;

decimal effectiveFee =
    fee ?? 0m;
```

Result:

EXAMPLE / FLOW

```
0
```

But explain:

EXAMPLE / FLOW

```
This is correct only if
missing fee means zero fee.
```

✦ Practical Exercise — Nullable Reference

C#

```
string? campaignName =
    null;

string display =
    campaignName
    ??
    "No Campaign";
```

Output:

EXAMPLE / FLOW

```
No Campaign
```

✦ Practical Exercise — Null-Conditional

C#

```
Campaign? campaign =
    null;

string? name =
    campaign?.Name;
```

Result:

EXAMPLE / FLOW

```
null
```

No member dereference occurs.

✦ Practical Exercise — Chain

C#

```
string city =
    gift
        ?.Constituent
        ?.PrimaryAddress
        ?.City
    ??
    "Unknown";
```

Explain:

```
C#
Any optional reference in chain is null
    →
chain returns null
    →
?? selects "Unknown"
```

✦ Practical Exercise — Guard Clause

```
C#
static decimal GetGiftAmount(
    Gift? gift)
{
    if (gift is null)
    {
        throw new ArgumentNullException(
            nameof(gift));
    }

    return gift.TotalAmount;
}
```

After the null check:

```
EXAMPLE / FLOW
compiler knows gift is non-null
```

✦ Practical Exercise — Nullable bool

```
C#
bool? approved =
    null;
```

Write logic for:

```
EXAMPLE / FLOW
Approved
Rejected
Pending
```

Answer:

```C#
string status =
    approved switch
    {
        true
            => "Approved",

        false
            => "Rejected",

        null
            => "Pending"
    };
```

✦ Practical Exercise — Find Gift

```C#
Gift? gift =
    FindGift(
        1001);
```

If null is expected when not found:

```C#
if (gift is null)
{
    Console.WriteLine(
        "Gift not found.");

    return;
}
```

Then:

```C#
Console.WriteLine(
    gift.TotalAmount);
```

is safe according to the flow analysis.

✦ Interview Questions

What is null in C#?

`null` represents the absence of an object reference or nullable value.

It is different from:

EXAMPLE / FLOW

```
0

false

empty string
```

because those are actual values.

What is a nullable value type?

A nullable value type allows a normal non-nullable value type to also represent `null`.

Example:

C#

```
int? count =
    null;
```

`T?` for a value type is shorthand for:

EXAMPLE / FLOW

```
Nullable<T>
```

What is the difference between int and int??

`int` can contain integer values.

`int?` can contain:

EXAMPLE / FLOW

```
any int
or
null
```

Can a nullable type contain another nullable type?

No.

You cannot create:

C#

```
int??
```

as nested `Nullable<Nullable<int>>`.

The underlying `T` of `Nullable<T>` must itself be a non-nullable value type.

What does HasValue do?

`HasValue` tells you whether a `Nullable<T>` currently contains an underlying value.

EXAMPLE / FLOW

```
true
    →
value exists

false
    →
null
```

What does Nullable.Value do?

It returns the underlying value.

If no value exists, accessing `Value` throws `InvalidOperationException`.

What does GetValueOrDefault do?

It returns:

EXAMPLE / FLOW

```
contained value
```

when one exists.

Otherwise it returns either:

EXAMPLE / FLOW

```
default(T)
```

or a caller-specified fallback.

What are nullable reference types?

Nullable reference types are compiler annotations and static-analysis features that indicate whether a reference is intended to allow null.

Examples:

C#

```
string
```

means intended non-null,

while:

```C#
string?
```

means null is allowed.

Is string? a different runtime type from string?

No.

Both use the same CLR `System.String` type.

The `?` annotation provides compile-time nullability intent and analysis.

How do you enable nullable reference types?

Normally in `.csproj`:

```XML
<Nullable>enable</Nullable>
```

Recent .NET project templates normally enable nullable context by default.

What is null-state analysis?

The C# compiler tracks whether a reference expression is approximately:

```EXAMPLE / FLOW
not-null
```

or:

```EXAMPLE / FLOW
maybe-null
```

based on declarations and program flow.

This allows it to warn about potentially unsafe dereferences.

What is the difference between nullable annotation and null-state?

Annotation:

```EXAMPLE / FLOW
string?
```

describes the declared API intent.

Null-state:

```EXAMPLE / FLOW
maybe-null
or
not-null
```

describes what the compiler currently knows at a particular point in the program.

What does ?. do?

`?.` performs member access only when the left-side object is non-null.

If it is null, the expression returns null instead of accessing the member.

It short-circuits subsequent null-conditional accesses.

What does ?[] do?

It performs array/indexer access only when the target object is non-null.

It does not prevent exceptions from an invalid index when the object itself exists.

What does ?? do?

`??` returns the left operand if it is non-null.

Otherwise it evaluates and returns the right operand.

Example:

```C#
string name =
    nullableName
    ??
    "Unknown";
```

What does ??= do?

`??=` assigns the right-hand value only when the current left-hand value is null.

Example:

```C#
items ??=
    new List<Gift>();
```

The right side isn't evaluated when `items` already contains a value.

What does the null-forgiving operator ! do?

The postfix `!` suppresses nullable compiler warnings for the expression.

It tells the compiler to treat the value as non-null.

It does **nothing at runtime** and cannot prevent `NullReferenceException`.

Should you use ! to fix nullable warnings?

Not automatically.

Use it only when you genuinely know something the compiler cannot prove.

Otherwise:

EXAMPLE / FLOW

```
handle null
or
correct the model
```

instead of suppressing the warning.

What is the difference between is null and == null?

Both commonly test for null.

However:

C#

```
is null
```

uses pattern-matching null semantics and isn't affected by an overloaded equality operator.

It is therefore a very explicit null test.

Can a non-nullable reference still be null at runtime?

Yes.

Nullable reference types are static-analysis annotations rather than a new runtime representation.

Broken contracts, reflection, deserialization, legacy libraries, interop, or misuse of `!` can still introduce runtime null values.

What is a lifted operator?

An operator defined for a value type can be lifted to its nullable form.

Example:

C#

```
int? a = 10;
int? b = null;

int? result =
    a + b;
```

typically produces:

EXAMPLE / FLOW

```
null
```

because one operand is absent.

What is NotNullWhen?

`NotNullWhen` is a nullable static-analysis attribute.

It tells the compiler that a parameter is guaranteed non-null when the method returns a particular Boolean result.

Example use:

EXAMPLE / FLOW

```
Try-pattern
```

or custom null-check methods.

What is the difference between nullable value types and nullable reference types?

Nullable value type:

C#

```
int?
```

is:

EXAMPLE / FLOW

```
Nullable<int>
```

and is an actual value-type representation capable of storing an integer or null.

Nullable reference type:

C#

```
string?
```

is the same runtime reference type as `string`, with compiler annotations describing nullability intent.

✦ Best Practices

Enable nullable analysis in modern .NET projects.

Use:

C#

```
T
```

when null is not a valid state.

Use:

C#

```
T?
```

when absence is genuinely meaningful.

Do not make everything nullable merely to avoid compiler warnings.

Do not mark legitimately optional values as required.

Treat nullable warnings as useful design feedback.

Prefer:

```
C#
is null
```

and:

```
C#
is not null
```

for clear null-state checks.

Use:

```
C#
?.
```

for legitimate optional navigation.

Use:

```
C#
??
```

only when the fallback has correct business meaning.

Use:

```
C#
??=
```

for clear lazy initialization where appropriate.

Use:

```
C#
!
```

sparingly.

Prefer actual initialization over:

```
C#
= null!;
```

when you control object construction.

Avoid accessing:

```
C#
nullable.Value
```

without proving a value exists.

Use pattern matching to safely extract nullable values.

Keep:

EXAMPLE / FLOW

```
missing
zero
false
empty string
```

as separate states when the business meaning differs.

Make method parameters and return annotations accurately describe their contracts.

Use nullable analysis attributes only when simple annotations cannot describe the API contract adequately.

Remember that nullability is not complete validation.

For security-sensitive values, don't use permissive fallbacks that accidentally grant access.

✦ Quick Revision

Concept	Meaning
`null`	Absence of reference/value
`int?`	Nullable integer
`Nullable<T>`	Underlying nullable value-type structure
`HasValue`	Whether nullable value exists
`Value`	Extract underlying nullable value
`GetValueOrDefault`	Value or fallback
`string?`	Reference intended to allow null
`string`	Reference intended to be non-null
Null-state	Compiler's current knowledge of null possibility
`is null`	Explicit null pattern check
`is not null`	Explicit non-null pattern check
`?.`	Null-conditional member access
`?[]`	Null-conditional index access
`??`	Fallback when null
`??=`	Assign only when null
`!`	Suppress nullable warning
`ThrowIfNull`	Reject invalid null argument
Lifted operator	Value-type operator applied to nullable values
`NotNullWhen`	Conditional compiler null-state contract
`AllowNull`	Allows null as API input precondition
`DisallowNull`	Declares null invalid as input
`MaybeNull`	Result may be null
`MemberNotNull`	Method establishes member as non-null

✦ Complete Fundraising Example

C#

```
public sealed class Campaign
{
    public required string Name
    {
        get;
        init;
    }
}

public sealed class Gift
{
    public long GiftId
    {
        get;
        init;
    }

    public long ConstituentId
    {
        get;
        init;
    }

    public long? CampaignId
    {
        get;
        init;
    }

    public Campaign? Campaign
    {
        get;
        init;
    }

    public decimal TotalAmount
    {
        get;
        init;
    }

    public string CurrencyCode
    {
        get;
        init;
    } = "USD";

    public string? Notes
    {
        get;
        init;
    }

    public DateTime? RefundedAtUtc
    {
        get;
        init;
    }
}

static string GetCampaignDisplay(
```

```
    Gift gift)
{
    ArgumentNullException.ThrowIfNull(
        gift);

    return gift.Campaign?.Name
           ??
           "No Campaign";
}

static string GetRefundStatus(
    Gift gift)
{
    return gift.RefundedAtUtc
        is DateTime refundedAt

        ? $"Refunded at {refundedAt:u}"

        : "Not Refunded";
}
```

Usage:

C#

```
Gift gift =
    new Gift
    {
        GiftId =
            1001,

        ConstituentId =
            5001,

        CampaignId =
            null,

        Campaign =
            null,

        TotalAmount =
            1000m,

        CurrencyCode =
            "USD",

        Notes =
            null,

        RefundedAtUtc =
            null
    };

string campaign =
    GetCampaignDisplay(
        gift);

string notes =
    gift.Notes
    ??
    "No notes";

string refundStatus =
    GetRefundStatus(
        gift);
```

Result:

EXAMPLE / FLOW

```
Campaign
    →
No Campaign

Notes
    →
No notes

Refund Status
    →
Not Refunded
```

✦ Complete Nullability Decision Flow

EXAMPLE / FLOW

```
Business Value
    →
Can value legitimately be absent?
    →
No
    →
Use Non-Nullable Type
    →
Initialize correctly
    →
Compiler expects non-null state
```

If absence is valid:

EXAMPLE / FLOW

```
Business Value
    →
May be absent
    →
Value Type?
        →
        T?

Reference Type?
        →
        T?
```

Then:

EXAMPLE / FLOW

```
Nullable Value
    →
Need to use it
    →
Choose behavior
```

Possible behaviors:

EXAMPLE / FLOW

```
Check

is null
is not null
HasValue
pattern matching
```

or:

EXAMPLE / FLOW

```
Safely navigate

?.
?[]
```

or:

EXAMPLE / FLOW

```
Provide fallback

??
```

or:

EXAMPLE / FLOW

```
Lazy initialize

??=
```

or:

EXAMPLE / FLOW

```
Reject invalid absence

ThrowIfNull
throw expression
```

or rarely:

EXAMPLE / FLOW

```
Compiler cannot prove known invariant

!
```

✦ Complete Fundraising Flow

EXAMPLE / FLOW

```
Gift Request
    →
Required Constituent?
    →
Yes
    →
non-null / required identifier

Optional Campaign?
    →
Yes
    →
long? CampaignId
    →
Campaign? reference

Optional Notes?
    →
string?

Refund completed?
    →
No
    →
RefundedAtUtc = null

Later refunded
    →
RefundedAtUtc = timestamp
```

Reading the gift:

EXAMPLE / FLOW

```
Gift
    →
Campaign?.Name
    →
Campaign exists?
    →
Yes → Name
    →
No → null
    →
?? "No Campaign"
```

The main lesson is:

☀ **Nullability is part of your application design. `null` should have an intentional business meaning, nullable annotations should accurately describe your contracts, and compiler warnings should help you discover places where your code hasn't correctly handled absence.**

Do not aim to eliminate every `null` from a system.

Aim to make every legitimate `null` **intentional, understandable, and safely handled**.

Next roadmap topic: `const`, `readonly`, `static`, and Object Lifetime.